



AWS Lambda : Serverless Function

AWS Lambda is a serverless compute service that lets you run code without provisioning or managing servers. It executes your code only when needed and scales automatically, from a few requests per day to thousands per second. You pay only for the compute time you consume—there is no charge when your code is not running.

Key Concepts

Functions

A Lambda function is the code you run on AWS Lambda.

Concept	Description
Runtime	The language environment in which your function runs (e.g., Node.js, Python, Java)
Handler	The method in your code that processes events
Deployment Package	A ZIP file or container image containing your function code and dependencies
Memory Allocation	The amount of memory assigned to your function. Also determines the allocated CPU power

Triggers and Events

A Lambda function is invoked in response to an **event**. An event is a JSON document that contains data for the function to process.

Lambda functions can be triggered by various AWS services:

- **Data Stores:** Amazon S3, Amazon DynamoDB, Amazon RDS
- **API Gateway:** HTTP requests
- **Streaming Services:** Amazon Kinesis, Amazon SQS
- **Time-based:** Amazon EventBridge (formerly CloudWatch Events)
- **Other Compute:** Other Lambda functions

Setting Up Your First Function

1. **Choose a Runtime:** Select the language you want to use.
2. **Write Your Code:** Implement the handler logic.
3. **Configure IAM Role:** Lambda requires an Identity and Access Management (IAM) role to execute and access other AWS resources like logs in CloudWatch. The role should be called `<span type="placeholder" placeholder-type="person"></span>-Lambda-Execution-Role`.
4. **Deploy the Function:** Upload your code (deployment package). You can do this via the AWS Console, AWS CLI, or Serverless Application Model (SAM).

For a hands-on tutorial, refer to the official AWS Lambda Documentation: [📄 File](#)

Advanced Configuration

Environment Variables

You can use environment variables to pass configuration settings to your function code and libraries. They are recommended for settings that change based on the deployment stage (e.g., development, staging, production) or for non-sensitive configuration data.

Cold Starts and Provisioned Concurrency

A **cold start** occurs when Lambda needs to fully initialize the execution environment, which can add latency.

Provisioned Concurrency allows you to pre-warm execution environments, eliminating cold starts for a defined number of function instances.

Layers

Lambda Layers allow you to package and manage dependencies, custom runtimes, and other libraries separately from your main function code.

Layer Component	Typical Use Case
Dependencies	Storing common third-party libraries (e.g., Pandas for Python)
Custom Runtime	Running code in a language not natively supported by Lambda
Configuration Files	Storing common files accessed by multiple functions

Monitoring and Troubleshooting

AWS Lambda integrates seamlessly with Amazon CloudWatch for monitoring and logging.

CloudWatch Metrics

Key metrics to monitor for Lambda function health:

- **Invocations:** The number of times your function was invoked.
- **Duration:** The time your function code spent executing.
- **Errors:** The number of invocations that resulted in a function error.
- **Throttles:** The number of times the function invocation was limited by the concurrency limit.

CloudWatch Logs

All output from your function (e.g., print statements) is automatically logged to a CloudWatch log group named `/aws/lambda/<function-name>`.

For assistance with specific errors, please schedule a session with a solutions architect:

 [Calendar event](#)

Next Steps

1. **Explore the Serverless Framework:** Investigate frameworks like Serverless and AWS SAM to streamline development and deployment.
2. **Security Review:** Review the IAM role permissions and ensure the Principle of Least Privilege is followed.
3. **Cost Optimization:** Monitor memory usage and execution time to find the optimal configuration for performance and cost.

This guide provides a comprehensive overview of the core components of AWS Lambda. For deployment, we recommend using the `sam deploy` command from your project directory at `Place`.